AISYS | RFID-Enabled Library Management System

FINAL TECHNICAL REPORT | Candidate technical evaluation | 09 November 2026

Evidence-based prototype, implementation boundaries and production design

Repository baseline: 66b870c • Source: Phase 1 requirements, Phase 2 design and Phase 3 recorded verification

Executive summary

This submission documents a local RFID-enabled library management prototype and its broader proposed architecture. Implemented components are React/TypeScript staff screens, FastAPI backend, SQLAlchemy/SQLite persistence, Alembic migrations, Argon2 authentication, revocable bearer-token sessions, permission-based access controls, list/create catalogue and member APIs, checkout/return, selected audit events and mock EPC assignment/scan. All demonstration records are synthetic.

Recorded execution evidence shows 19 passing backend Pytest tests, one non-failing deprecation warning, successful React production build, successful Alembic checks, browser-originated API workflow and repeatable synthetic seed. These are Phase 3 recorded results, not independent test reruns for Phase 4.

The system is not a production-certified ILMS or RFID integration. No real RFID devices, NCIP/SIP2 connector, existing ILMS, offline sync, deployed PostgreSQL or production load tests are claimed.

1 | Problem, objectives and scope

Objective: demonstrate a controlled catalogue → physical copy → member → circulation journey and a device-neutral RFID simulation, while documenting requirements and safe interoperability boundaries. The full source baseline contains 12 functional requirements, 9 nonfunctional requirements and 10 acceptance criteria; the prototype delivers only a subset.

- Use synthetic data; never connect to or write into an institutional ILMS without authorization.
- Enforce access controls on backend endpoints, not solely in the browser.
- Separate verified features from proposals, mock boundaries and unimplemented scope.
- Deliver working setup instructions, reproducible evidence and a production gap register.

2 | Requirement traceability and acceptance

The accompanying AISYS_Phase4_Canonical_RTM.xlsx supersedes the Phase 1 progress tracking view for Phase 4 status; the original Phase 1 workbook remains unchanged as historical baseline. Status labels apply to complete source requirements, not isolated subfeatures. “Implemented” requires the full requirement to be supported; several functioning subfeatures remain “Partially implemented”.

Classification	Meaning
Implemented	Entire specified requirement demonstrably delivered and verified
Partially implemented	At least one component running; other stated parts missing or unproven

Design-only	Design or proposed contract exists, without implementation evidence
Unimplemented	No implementation and no corresponding developed design evidenced

Requirement-level coverage: Implemented: 0, Partially implemented: 13, Design-only: 8, Unimplemented: 0. No combined AC01–AC10 acceptance scenario can be signed off as fully met using the available evidence; AC02, AC03, AC04 and AC07 have partial demonstrations.

3 | Architecture and technical decisions

Implemented developer environment: React + TypeScript browser ↔️ FastAPI authenticated REST endpoints ↔️ SQLAlchemy data access ↔️ SQLite, with Alembic migrations. Mock RFID APIs resolve synthetic EPCs to local physical items. Backend includes user sessions, permission checks and selected audit events.

Proposed production architecture: a modular monolith, PostgreSQL database, isolated LAN/TLS proxy, device-neutral adapters for reader/gate/smart card, NCIP 2.0 and SIP2 ports, controlled ILMS interface, durable outbox and retry worker. None of these production integration elements is verified by a deployed system.

Layer	Prototype verified	Proposed / not implemented
UI	React basic staff workflows	Full OPAC, RFID management UI, advanced search
API	FastAPI auth/catalog/items/members/loans/mock RFID	Other Phase 2 route contracts
Persistence	SQLite, SQLAlchemy, Alembic	PostgreSQL deployment, full migrations/data reconciliation
Device boundary	Synthetic EPC assign and scan	Physical RFID readers, gate, smart cards
Operations	Local runbook and health checks	Offline installers, secure production hosting, backup/restore proof

Representative implemented flow

An administrator signs in, adds bibliographic metadata, creates an item with unique accession, registers a synthetic member, checks an item out, verifies CHECKED_OUT state, returns the loan, then verifies AVAILABLE. RFID assign and scan are separate Swagger API interactions, returning hardware_connected=false. Checkout/return do not automatically arm/disarm the RFID security bit.

4 | Data model and integrity

The logical design covers bibliographic records, physical copies, members, loans, user/role/permission, sessions, RFID tags, audit events and proposed inventory, gate and migration records. Live prototype schema details must be treated as defined by Alembic and SQLAlchemy models; the Phase 2 data model contains further future tables and constraints that are not necessarily present in SQLite.

- Bibliographic record describes a title; a physical item has accession and circulation state.
- Loan links item and member; returns restore item availability in the demonstrated flow.
- RFID EPC is synthetic and unique within the tested mock assignment flow.
- Production requirements include active-loan concurrency guards, durable event outbox, tag history, staging reconciliation and restore validation; these are not fully implemented.

5 | REST interface and permission model

The verified application provides authentication, list/create bibliographic records, list/create physical items, member registration/list, checkout, return, and mock tag assign/scan. Exact live paths and request schemas should be reviewed in local FastAPI Swagger at http://127.0.0.1:8000/docs; the Phase 2 rest-api-contracts.md also includes unimplemented proposal endpoints and is not an as-built OpenAPI record.

API demonstration	Evidence boundary
Login/logout and permissions	Tests document authentication and RBAC baseline
Catalogue, items and members	Create/list covered in Phase 3 summary
Circulation checkout/return	Both demo flow and Pytest coverage
POST /api/v1/rfid/tags/assign	Tested software-only association
POST /api/v1/rfid/scans/mock	Tested lookup and unknown-tag behavior
Renewals, gates, reporting, migration	No verified implementation in uploaded evidence

6 | Security, privacy and compliance considerations

Prototype security mechanisms: Argon2 password hashes, revocable bearer sessions, permission-based endpoint checks, synthetic data and selected audit entries. Local-process rate limiting is not distributed; database-enforced audit immutability is absent. TLS, secrets management, monitoring and penetration testing need production validation. Prevent .env, local databases, secrets and credentials from entering Git or the submission package.

7 | RFID and integration boundary

Implemented RFID behavior is deliberately a software simulation: bind a unique mock EPC to an existing item and resolve EPC using the scan route. This validates part of application logic but not physical read reliability, tag programming, electromagnetic operation, SDK compatibility or gate triggers. Vendor-neutral NCIP/SIP2 ports, ILMS integration and device security-state synchronization remain design-only/unimplemented.

8 | Circulation policies and gaps

Checkout/return are implemented. A reference-copy restriction is shown in recorded test names. The source requires renewals, due dates, blocked-member and fine-limit policy, smart-card personalization and full audit/protocol behavior; these have not all been demonstrated. A transaction success in the mock local database must not be described as an NCIP/SIP2-certified transaction.

9 | Migration and data protection

The Phase 1 baseline calls for approximately 20,000 spreadsheet rows with validated staging, deduplication, error reporting, reconciliation and rollback. Design invariants include source_count = accepted_count + rejected_count + duplicate_count and distinct insert/update accounting. No real or synthetic 20k import, error log, rollback or checksum-based restore proof is included in the Phase 3 evidence. This remains an acceptance blocker.

10 | Verification method and results

Check	Recorded outcome	Source
Backend Pytest	19 passed, 1 warning	evidence/phase3/pytest-results.txt

React build	tsc -b and vite build succeeded	evidence/phase3/frontend-build.txt
Database	Alembic migration checks recorded	evidence/phase3/alembic-check.txt
Browser API	Login, create records, checkout, return	evidence/phase3/ implementation-verification.md
Seed repeat	Second run created no additional records	evidence/phase3/ implementation-verification.md

The provided ZIP contains captured outputs rather than the application source tree. Therefore this report is anchored in recorded results, not a source-code audit or current execution. Before submission rerun automated checks in the full repository and record new results if anything changes.

11 | Risk register and production-readiness gaps

Risk / open work	Severity	Recommended follow-up
Legacy records modification	Critical	Read-only contract, sandbox migration, backup/checksum restore
Misstated RFID certification	High	Vendor SDK, real hardware and protocol tests
Loan concurrency / event loss	High	PostgreSQL transactions; durable outbox/idempotency
Offline deployment failure	High	Disconnected install-update-rollback drill
Migration loss or duplication	High	Staging, reconciliation and rollback proof
Security hardening	High	TLS, secrets, distributed throttling, audit immutability
Reporting and inventory absent	Medium	Implement prioritized user stories and tests

12 | Deployment and reproduction

Local Windows reproduction requires Python 3.13, Node/npm and Git. From backend: create .venv using py -3.13 -m venv .venv; install requirements; copy .env.example to .env; run alembic upgrade head; run scripts.seed_auth and scripts.seed_library; start uvicorn app.main:app --reload. From frontend: npm ci; copy .env.example .env; npm run build; npm run dev. See docs/implementation/phase3-prototype-runbook.md for commands and login/demo sequence.

Verify with backend python -m pytest -v, alembic check and pip check; frontend npm run build. FastAPI Swagger is at 127.0.0.1:8000/docs and frontend dev server at localhost:5173. Local SQLite and credentials should remain excluded from Git and archived submission files.

13 | Staged roadmap

Stage	Required follow-up
Immediate demo readiness	Validate branch and current test outputs; capture UI screenshots; correct stale README status
Next engineering iteration	Complete renewal and lending restrictions, tag security sync and frontend RFID screens
Integration iteration	Real RFID adapter / mock gate, NCIP/SIP2 contract verification, notifications
Operational iteration	20k migration, inventory and reports, offline packaging, restore drill

Production qualification	PostgreSQL, concurrency/load/security/UAT, vendor certification, controlled pilot

14 | Submission inventory and limitations

Package includes this report (DOCX and PDF), a 16-slide presentation, an evidence-based canonical RTM, a submission README, and a Phase 4 repository-update patch directory. Original Phase 1–3 source documents remain in the Git repository. This ZIP does not include application source or credentials; the live code must be obtained from the repository.

Important: Source AISYS SOP PDF itself is not included in the uploaded source ZIP. Requirements here are based on the repository's Phase 1 SOP-derived summaries. External evaluator review and signoff are outstanding. The final Git commit/push are user-run steps, not performed by this environment.

Appendix | Source document register

- docs/requirements/AISYS_Phase1_Requirements_Initiation.docx
- docs/requirements/AISYS_Phase1_RTM_Progress.xlsx
- docs/architecture/system-architecture.md
- docs/database/data-model.md
- docs/api/rest-api-contracts.md
- docs/integrations/integration-design.md
- docs/security/security-and-business-rules.md
- docs/implementation/phase3-prototype-runbook.md
- docs/implementation/phase3-traceability-summary.md
- evidence/phase3/implementation-verification.md
- evidence/phase3/pytest-results.txt
- evidence/phase3/frontend-build.txt